https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

StaffManager - API на Django для платформы управления персоналом

Цель проекта Разработка корпоративной системы управления персоналом для автоматизации кадрового учёта, управления рабочими местами и компетенциями сотрудников. Система обеспечивает централизованное хранение профилей, расчёт стажа, управление навыками, разграничение прав доступа и интеграцию с внешними сервисами через REST API

Ключевые технические особенности

- **Бэкенд и архитектура:** Python 3.12, Django 5.2.1. Приложения staff и workplaces. Связь пользователя и сотрудника через OneToOneField. Связь сотрудник –навыки «многие-ко-многим» через промежуточную модель SkillLevel с валидацией уровня навыков в диапазоне от 1 до 10.
- **БД и оптимизация:** PostgreSQL и SQLite, prefetch_related. Настройка on_delete=models.SET_NULL для сохранения профиля сотрудника при удалении рабочего места.
- **Бизнес-логика и валидация:** валидация дат и правил рассадки сотрудников, расчёт стажа, управление главным изображением галереи.
- **Веб-интерфейс:** Class-Based Views (ListView, DetailView), защита через LoginRequiredMixin, авторесайз изображений (Pillow).
- **REST API:** Django REST Framework, ViewSet, ModelSerializer и вложенные сериализаторы. CRUD, пагинация и фильтрация сотрудников по навыкам и стажу. Документация Swagger/ReDoc (drf-yasg).
- **Безопасность:** JWT-аутентификация (SimpleJWT) с разграничением Customer, Supervisor, Admin.
- **Тестирование и качество кода:** unit-тесты на unittest для проверки бизнес-логики, прав доступа, маршрутов и контекста шаблонов. Позитивные и негативные сценарии. Использование Black, isort и Django Debug Toolbar.
- **Деплой и инфраструктура:** Docker/docker-compose, два сценария развертывания с SQLite3 и PostgreSQL, отдельный контейнер БД для PostgreSQL, healthcheck зависимостей, entrypoint.sh с ожиданием PostgreSQL, автоматическим созданием БД и выполнением миграций. Конфигурация через .env.

Достигнутые результаты

1. **Управление персоналом:** CRUD для сотрудников, рабочих мест и навыков; массовый импорт и экспорт CSV/XLSX.
2. **Автоматизация учета:** расчёт стажа, проверка дат приёма, валидация рассадки и автоматическое назначение главного изображения в галерее.
3. **REST API:** Получение списка и профилей сотрудников, фильтрация, создание и изменение данных с учётом прав доступа.
4. **Визуализация и документация:** Интерактивная документация API (/swagger/ и /redoc/).
5. **Готовность к эксплуатации:** Настроен отказ от Debug-инструментов в продакшене, вынесены переменные окружения в .env, реализован healthcheck для базы данных в Docker. Проект разворачивается одной из команд

    - *docker-compose -f docker-compose_sqlite.yml up –d --build*
    - *docker-compose -f docker-compose_postgres.yml up -d --build.*

https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

ClockAlarm - Консольный будильник на Python

Цель проекта Создать консольное приложение-будильник с упором на **оптимизацию использования системных ресурсов**. Проект демонстрирует практическое применение стандартных библиотек Python (datetime, time) и сторонней библиотеки pygame для работы со звуком. Ключевой задачей было реализовать логику, при которой программа не потребляет ресурсы процессора в режиме ожидания.

Ключевые технические особенности:

- **Эффективное ожидание:** вместо постоянного опроса системного времени в цикле, что создавало бы нагрузку на CPU, приложение использует функцию time.sleep(). Программа переходит в спящий режим и «просыпается» ровно за 1 секунду до установленного времени будильника.
- **Гибкая настройка сигнала:** реализована возможность выбора типа побудки — беззвучный режим (с выводом текстового сообщения) или одна из предустановленных мелодий.
- **Интерактивное управление:** в момент звонка пользователь может отложить сигнал на 5 минут (до 5 раз подряд) или полностью его остановить.
- **Надёжность:** программа включает проверку вводимых пользователем данных на корректность (дата, время) и обработку исключений для предотвращения сбоев.

Достигнутые результаты

В рамках проекта был разработан будильник, который эффективно решает поставленные задачи:

1. **Реализован механизм «спящего режима»**, позволяющий приложению не использовать процессорное время во время ожидания. Это достигается расчётом интервала сна до секунды перед срабатыванием будильника.
2. **Обеспечена поддержка нескольких будильников.** Пользователь может последовательно установить несколько событий.
3. **Создана система управления сигналом**, позволяющая откладывать или останавливать будильник после его активации.
4. **Добавлена валидация ввода** для защиты от некорректно заданных дат и времени.

https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

Установка мелодии и работа будильника

```
(.venv) PS C:\Users\EvgenyMINI_S\PythonProjects\ClockAlarm> py -m clockalarm
pygame 2.6.1 (SDL 2.28.4, Python 3.13.7)
Hello from the pygame community. https://www.pygame.org/contribute.html
ВАРИАНТЫ ПОБУДКИ:
0 -  Беззвучная

1 -  Louis Armstrong - Go Down Moses

2 -  Matia Bazar - Vacance Romane

3 -  Fallout New Vegas Radio - Jazz Blues

№ варианта: 2
Вы выбрали Matia Bazar - Vacance Romane

Установите время будильника 2026-MM-DD hh:mm
месяц MM: 06
число DD: 15
час(24) hh: 20
минут mm: 19
Будильник будет установлен!

Добавить будильник?...введите 'yes'
Продолжить - нажмите ENTER
Ваш выбор:

alarm_time: 2026-06-15 20:19
Мелодия: Matia Bazar - Vacance Romane
Чтобы остановить этот будильник, введите 'stop'.
Чтобы отложить на 5 минут нажмите ENTER.

Этот будильник отложен в 1й раз

Чтобы остановить этот будильник, введите 'stop'.
Чтобы отложить на 5 минут нажмите ENTER.

Этот будильник отложен в 2й раз

Чтобы остановить этот будильник, введите 'stop'.
Чтобы отложить на 5 минут нажмите ENTER.
stop
Этот будильник остановлен
Чтобы остановить все будильники, введите 'stop'.
Чтобы продолжить, нажмите ENTER.

(.venv) PS C:\Users\EvgenyMINI_S\PythonProjects\ClockAlarm>
```

Установка сообщения в беззвучном режиме

```
(.venv) PS C:\Users\EvgenyMINI_S\PythonProjects\ClockAlarm> py -m clockalarm
pygame 2.6.1 (SDL 2.28.4, Python 3.13.7)
Hello from the pygame community. https://www.pygame.org/contribute.html
ВАРИАНТЫ ПОБУДКИ:
0 -  Беззвучная

1 -  Louis Armstrong - Go Down Moses

2 -  Matia Bazar - Vacance Romane

3 -  Fallout New Vegas Radio - Jazz Blues

№ варианта: 0
Вы выбрали Беззвучная

Задайте сообщение будильника или нажмите ENTER
Сообщение:
Ваше сообщение (по умолчанию):  Время вставать!
Установите время будильника 2026-MM-DD hh:mm
месяц MM:
```

https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

Веб-приложение «Личный дневник» на Flask

Цель проекта Создать полноценное веб-приложение с использованием фреймворка Flask, реализующее функционал личного дневника. Ключевой задачей было освоить архитектуру *MVC* (Модель-Представление-Контроллер) в контексте Flask, научиться работать с базами данных через SQLAlchemy и реализовать систему аутентификации пользователей с управлением сессиями.

Ключевые технические особенности:

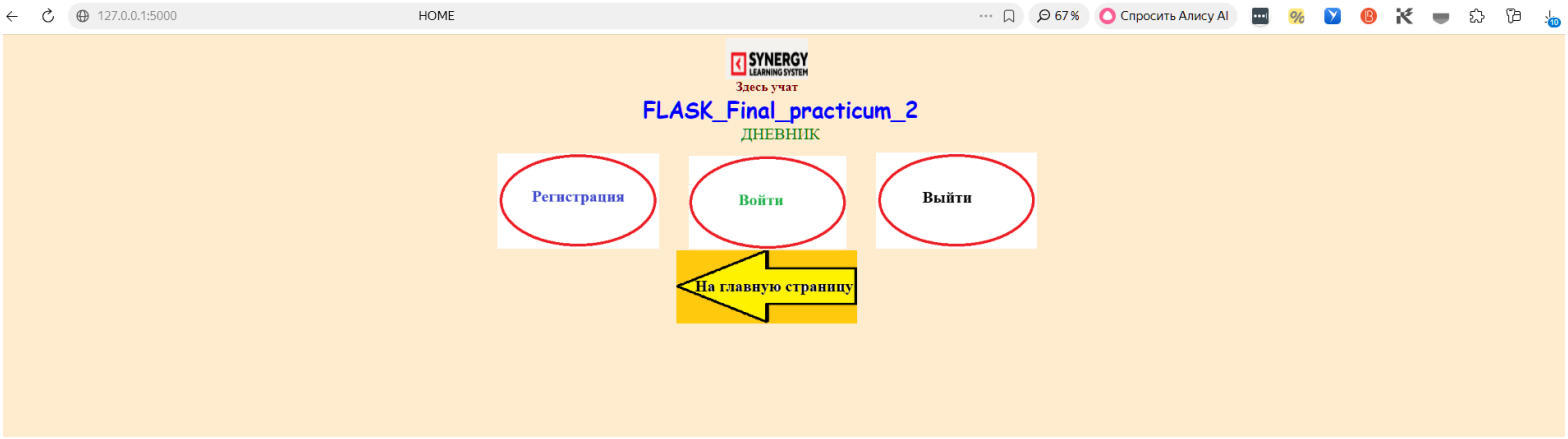
- **Бэкенд:** Разработка на Python с использованием микрофреймворка Flask.
- **База данных:** Использование SQLite для хранения данных. Управление схемой БД и миграциями осуществлялось с помощью Flask-Migrate.
- **Аутентификация:** Реализована система регистрации и входа пользователей с созданием сессий (токен действителен 30 минут). Доступ ко всем функциям приложения защищён и требует авторизации.
- **CRUD-операции:** Полный цикл операций над записями дневника: создание (*Create*), чтение (*Read*), редактирование (*Update*) и удаление (*Delete*).
- **Фронтенд:** Построение пользовательского интерфейса с помощью шаблонизатора Jinja2 и стилизация страниц с применением CSS.

Достигнутые результаты В рамках проекта был разработан прототип веб-сервиса, который позволяет пользователям:

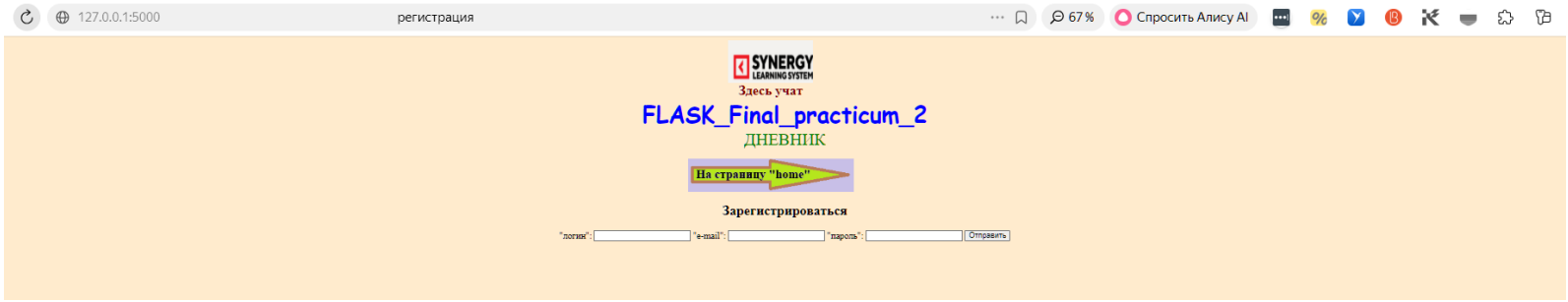
1. **Зарегистрироваться и войти в личный кабинет**, используя логин, email и пароль.
2. **Управлять личными записями**: создавать новые заметки, просматривать список всех записей, а также редактировать или удалять существующие по их уникальному идентификатору.
3. **Работать в безопасной сессии**. Все действия пользователя защищены, а время жизни сессии ограничено для обеспечения базовой безопасности.
4. **Использовать адаптивный интерфейс**. Приложение корректно отображается на различных устройствах благодаря использованию мета-тега viewport и продуманной верстке.

https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

1. ***Страница вход/выход/регистрация (/home)***



2. ***Страница регистрации (/register).***

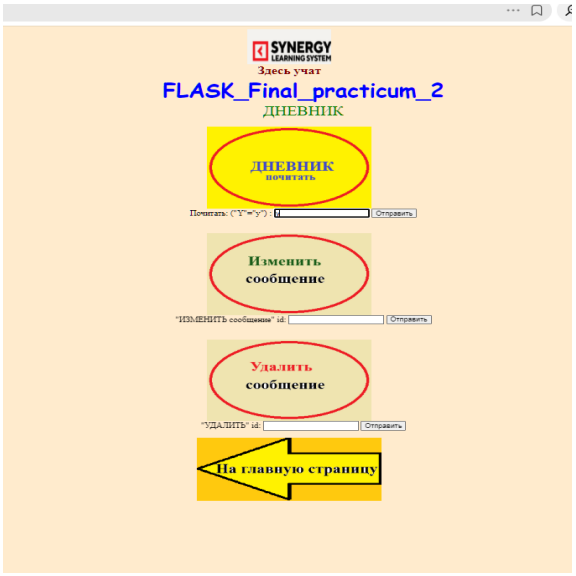


3. ***Основной экран дневника (/) - управление записями***

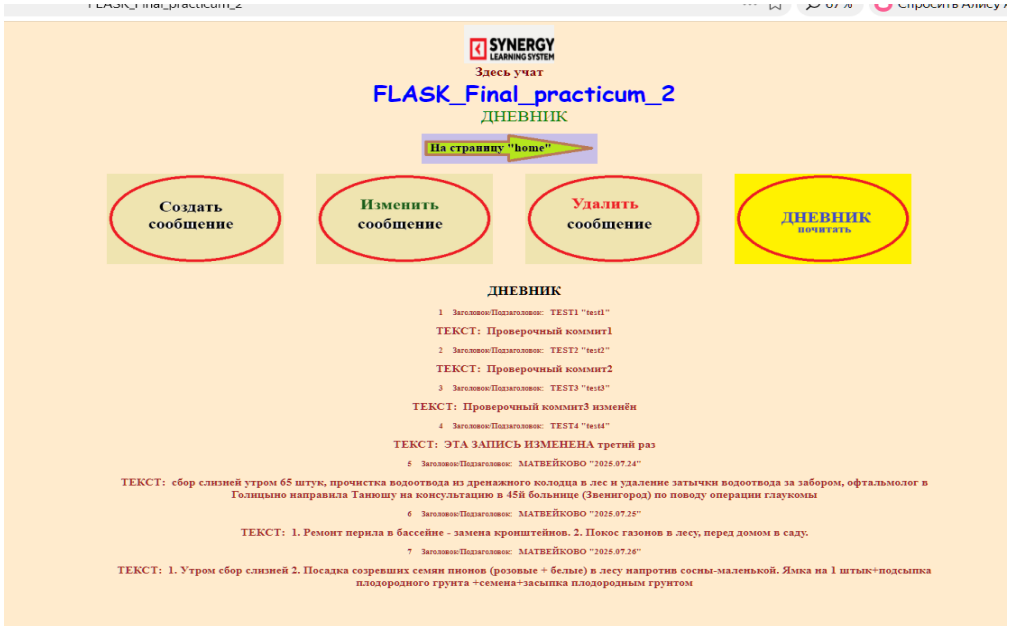


https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

4. *Страница выбора действий с записями (/card)*

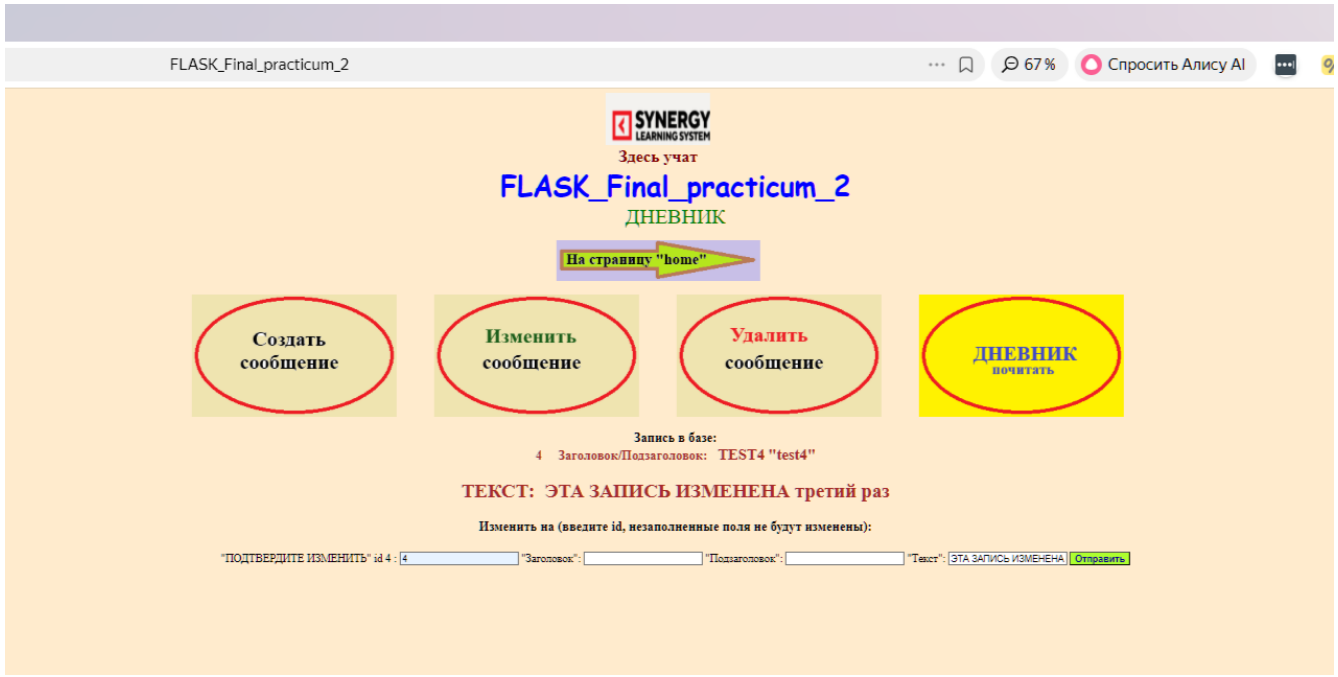


Почитать дневник

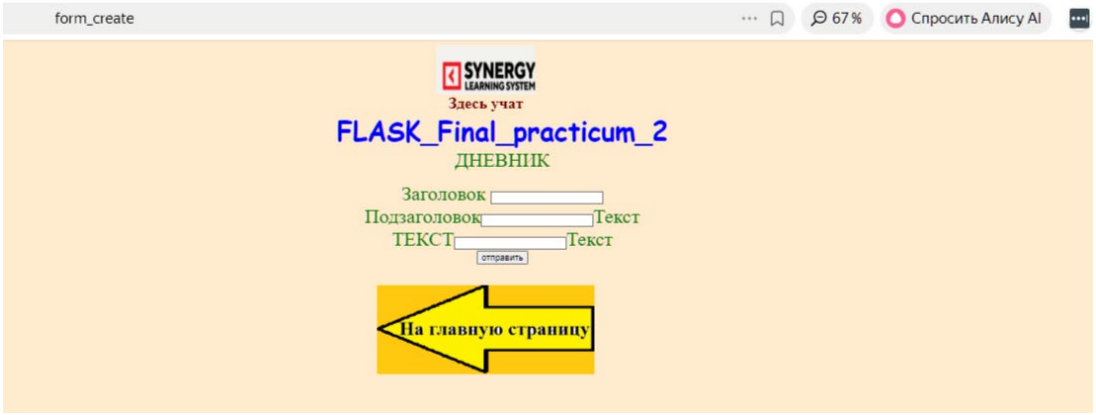


https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

ИЗМЕНИТЬ сообщение (изменение 4й записи).



5. ***Создать сообщение (/form_create)***



https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

DjangoMoneyCollect - REST API для групповых денежных сборов на Django

Цель проекта Разработать и реализовать полноценное REST API для платформы по организации групповых денежных сборов (краудфандинга). Проект нацелен на создание серверной части, которая предоставляет структурированный доступ к данным о сборах, платежах и пользователях, а также реализует бизнес-логику взаимодействия между ними.

Ключевые технические особенности:

- **Архитектура:** Проект построен на базе Django REST Framework (DRF), что позволило быстро создать стандартизированное и документированное API.
- **Асинхронность:** Для выполнения фоновых задач (отправка email-уведомлений) реализована интеграция с Celery и Redis, что обеспечивает высокую отзывчивость основного API. Так как API создавалось в среде Windows, то для совместимости Windows с Celery и эмуляции многопоточности был выбран асинхронный пул gevent с сопутствующим «monkey-патчированием» штатных библиотек.
- **Масштабируемость и развертывание:** Проект полностью готов к контейнеризации с помощью Docker и Docker-Compose, что упрощает его развертывание и масштабирование в различных средах.
- **Безопасность:** Реализована система авторизации и разграничения прав доступа к эндпоинтам API. Доступ к административным функциям ограничен.

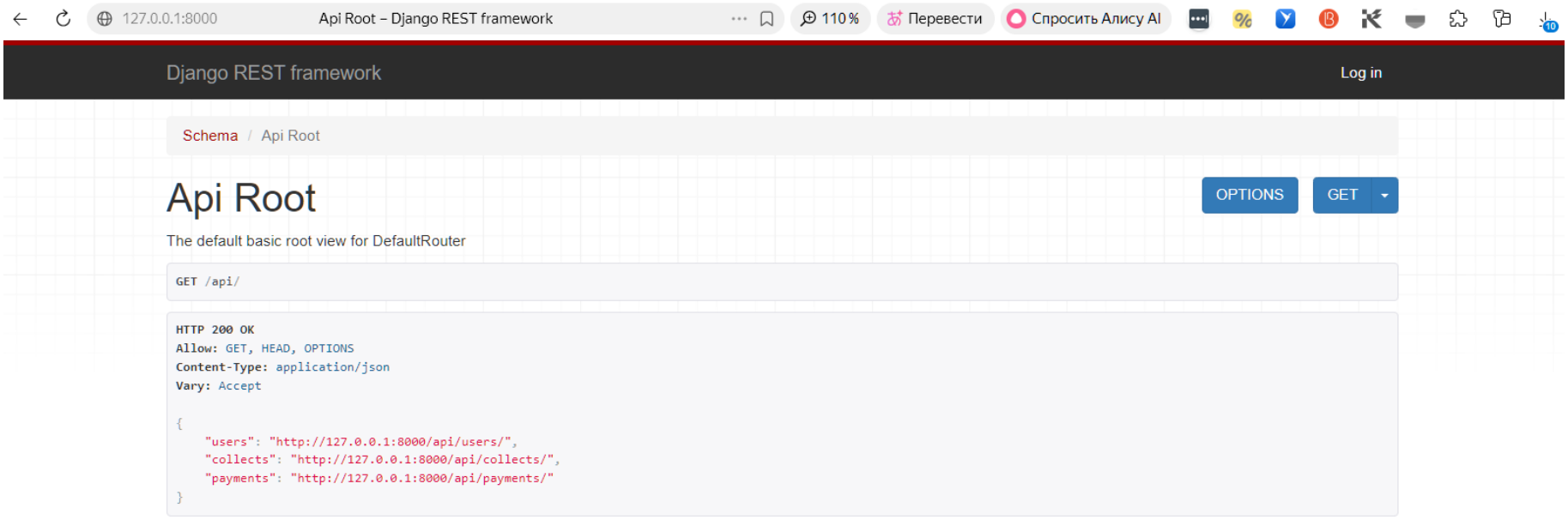
Достигнутые результаты В рамках проекта был разработан бэкенд, предоставляющий следующие возможности:

1. **Управление сборами:** Создание, чтение, обновление и удаление (CRUD) целей сборов. Доступен отдельный эндпоинт для получения списка активных сборов.
2. **Обработка платежей:** Реализован функционал для совершения пожертвований в пользу конкретного сбора и просмотра истории платежей.
3. **Управление пользователями:** Создание и просмотр пользователей системы.
4. **Гибкий поиск и фильтрация:** Клиенты API могут фильтровать сборы по категории (например, благотворительность), выполнять полнотекстовый поиск по названию и описанию, а также сортировать результаты.
5. **Автоматизация:** Настроена автоматическая отправка email-уведомлений участникам сборов при создании или обновлении событий.
6. **Тестовые данные:** Реализован инструмент Management command генерации моковых данных в режиме отладки (**generate_mock_data.py**) для быстрого наполнения базы данных большим количеством записей (пользователи , сборы, платежи…) с применением пакетных SQL-запросов.

https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

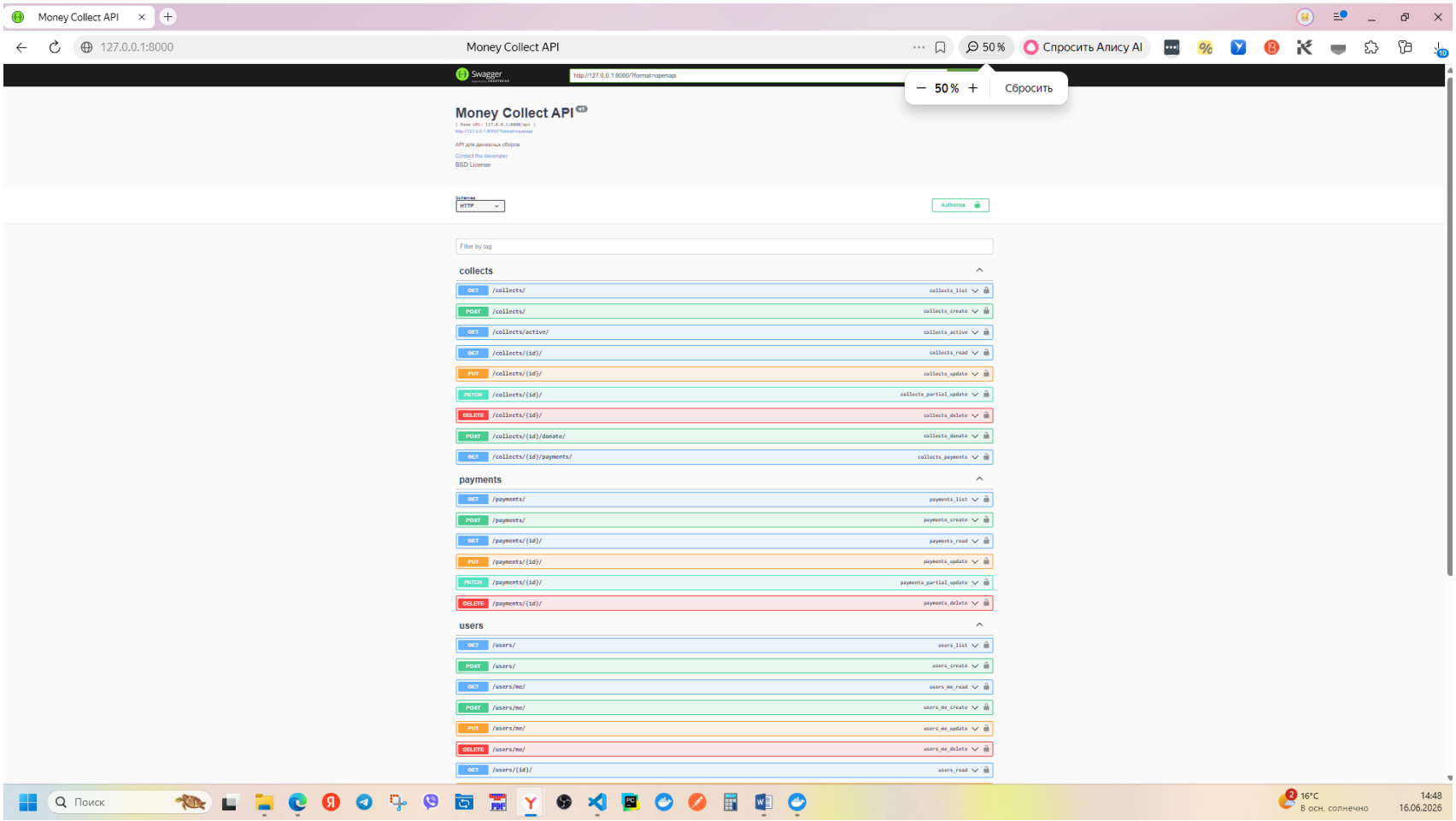
Демонстрация работы API.

1. Список доступных эндпоинтов API



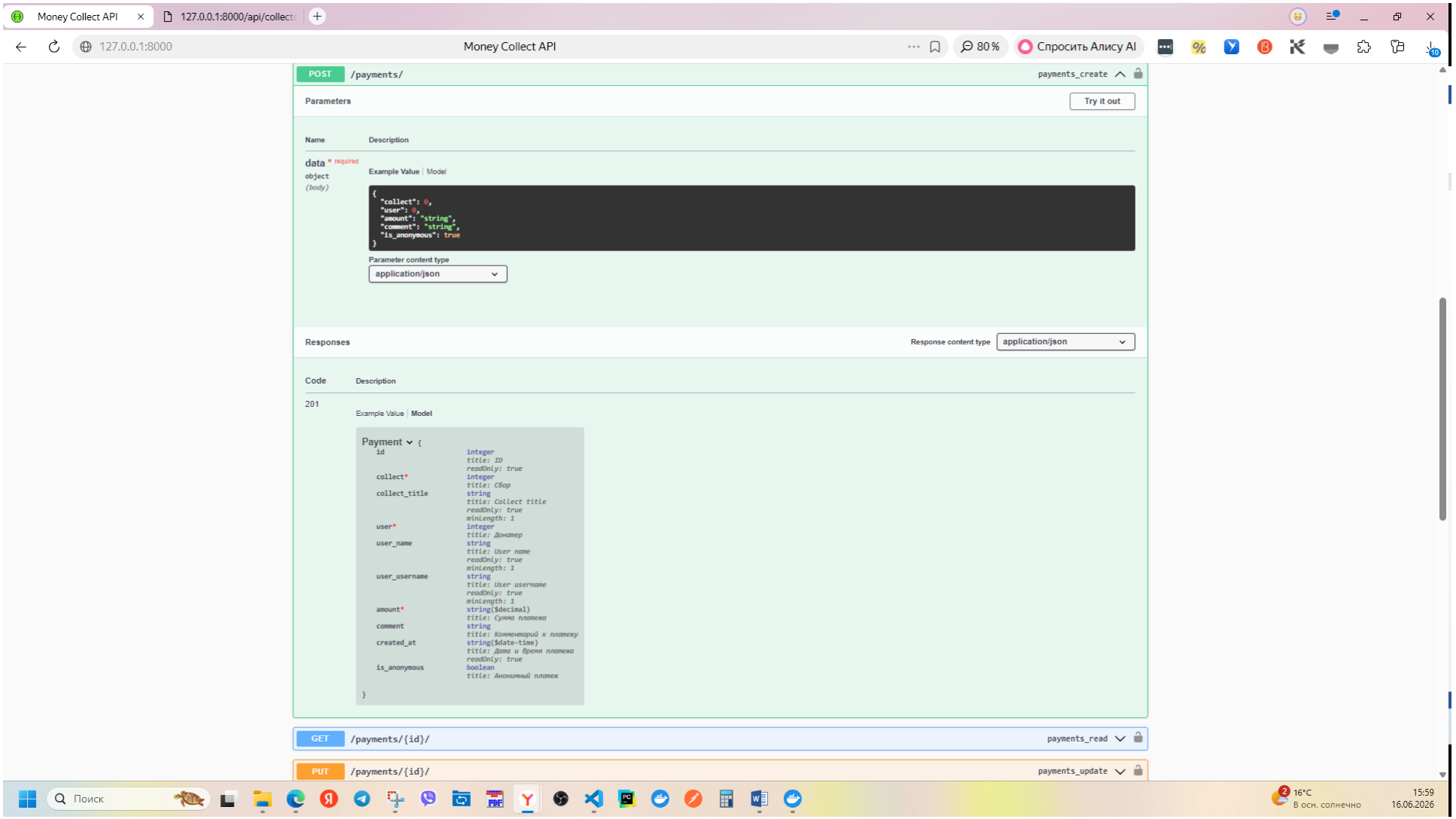
https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

2. *Главная страница SwaggerUI (/swagger/*



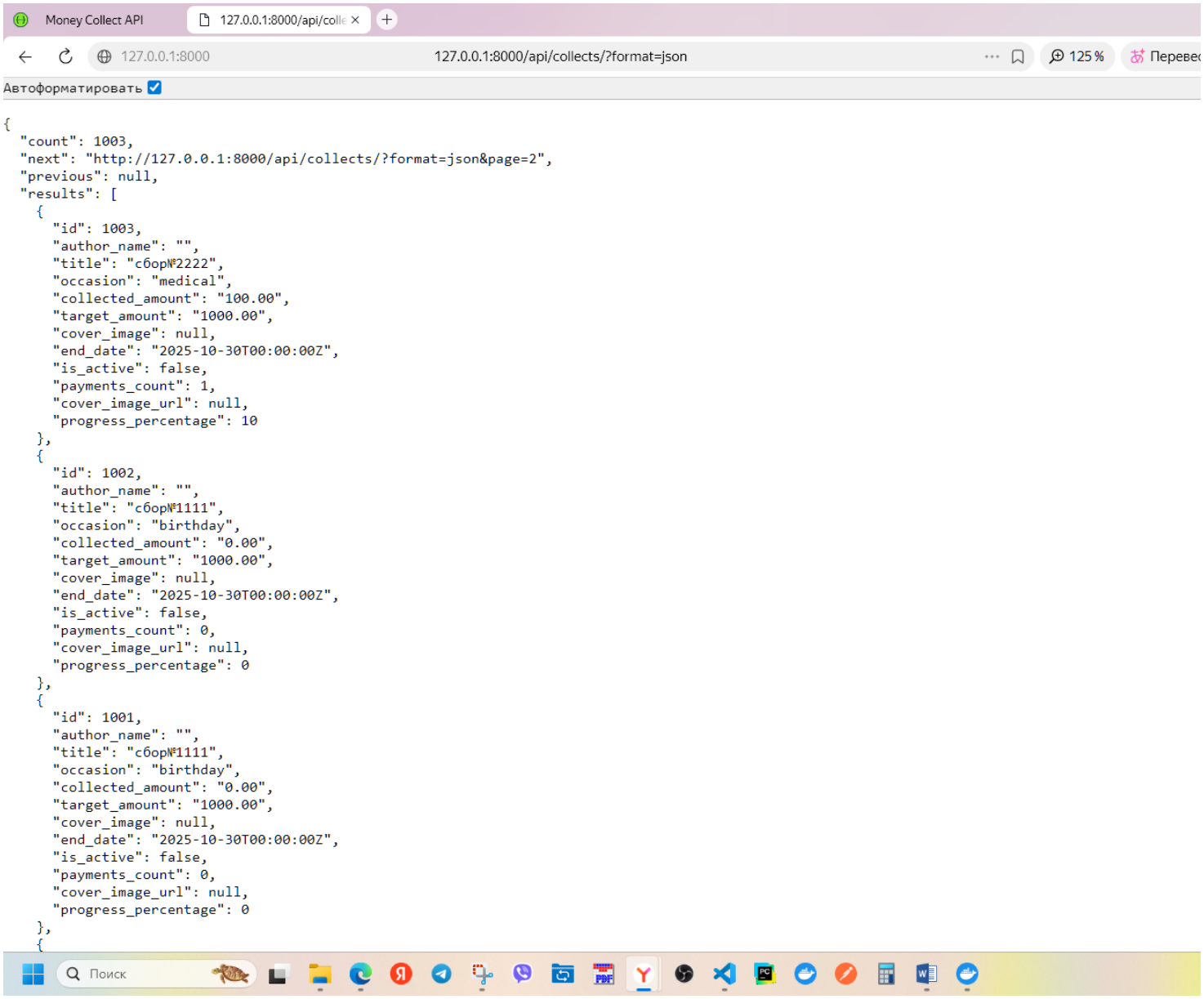
https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

3. *Документация эндпоинта создания взноса в сбор.*



https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

4. *Пример ответа API (список сборов).*



https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

DatingApp - API для платформы знакомств (DatingApp) на Django

Цель проекта Разработать REST API для веб-платформы знакомств, реализующий полный цикл взаимодействия между пользователями: от регистрации и поиска до общения в чатах. Ключевой задачей было создание сложной системы взаимодействий (лайки, дизлайки, мэтчи, приглашения) и обеспечение безопасной авторизации через различные каналы (email/пароль, социальные сети, JWT).

Ключевые технические особенности:

- **Архитектура:** Проект построен на Django с использованием Django REST Framework (DRF) для создания API. В качестве базы данных используется PostgreSQL.
- **Аутентификация:** Реализована гибкая система входа с поддержкой JWT-токенов (SimpleJWT) и OAuth2-авторизации через социальные сети (Google, Yandex, Mail.ru) с помощью пакета django-allauth.
- **Взаимодействия:** Сложная логика взаимодействий между пользователями: система лайков и дизлайков, формирование взаимных симпатий (мэтчей), история просмотров, приглашения на свидания и защищенный обмен контактами.
- **Реальное время:** Для реализации чатов между пользователями применена технология WebSocket через Django Channels.
- **Докеризация:** Проект готов к развертыванию в изолированной среде с помощью Docker и Docker Compose. Для удобства работы с базой данных в режиме отладки в стек включён pgAdmin 4, доступный по адресу http://127.0.0.1:5050 после запуска контейнеров.

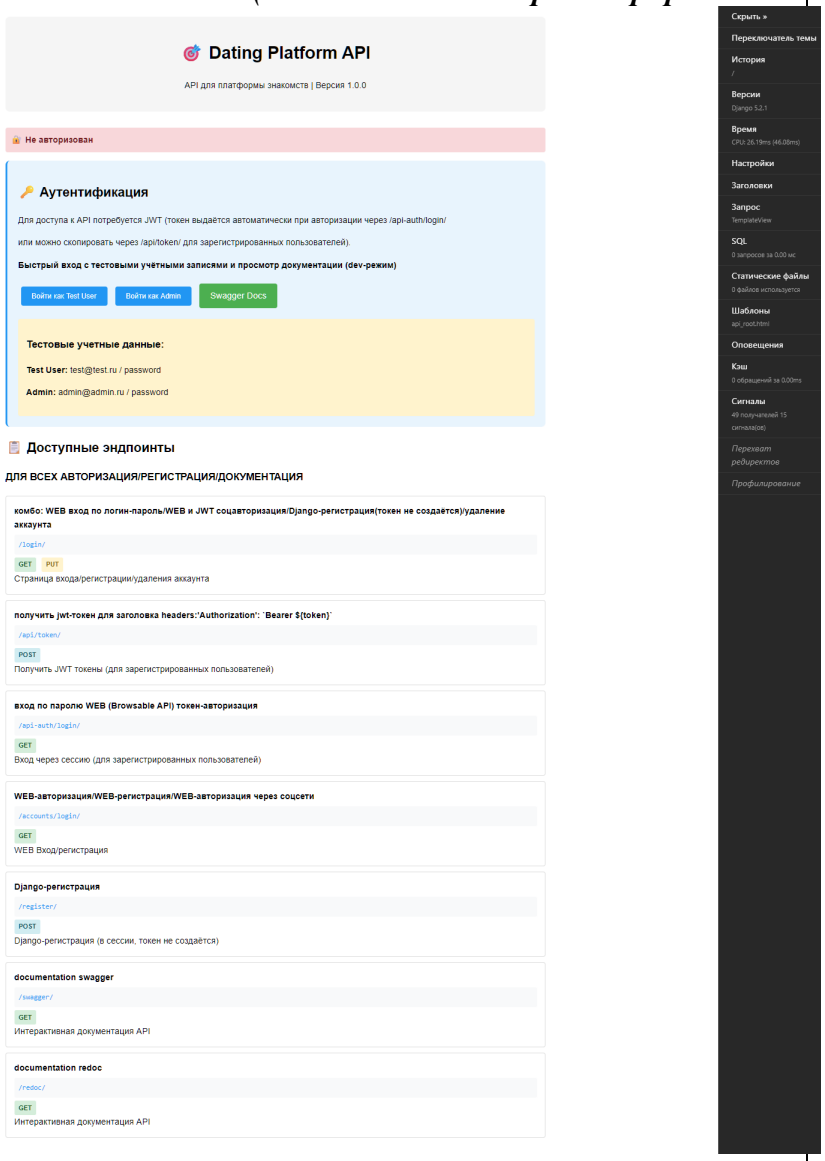
Достигнутые результаты В рамках проекта был разработан бэкенд, предоставляющий следующие возможности:

1. **Регистрация и авторизация:** Пользователи могут создавать аккаунты через email/пароль или входить с помощью учетных записей социальных сетей. При любом способе входа выдается JWT-токен для доступа к защищенным эндпоинтам.
2. **Система поиска и фильтрации:** Реализован механизм подбора случайных профилей с возможностью фильтрации по полу, возрасту, городу и статусу отношений.
3. **Механизм взаимодействий:** Пользователи могут ставить лайки и дизлайки другим профилям. При взаимном лайке формируется «мэтч», который открывает доступ к личному чату.
4. **Лента событий:** Доступна история просмотренных профилей, полученных лайков и отправленных дизлайков.
5. **Чаты (WebSocket):** Пользователи могут общаться в личных чатах со своими «мэтчами» в режиме реального времени с сохранением истории сообщений в базе данных.
6. **Приглашения и обмен контактами:** Реализована система отправки приглашений на свидание или предложения об обмене контактами, которая требует взаимного подтверждения для безопасности.
7. **Генерация тестовых данных:** Реализован инструмент Management command генерации моковых данных в режиме отладки с применением пакетных SQL-запросов (**generate_mock_data.py**) для быстрого наполнения базы данных большим количеством пользователей и их взаимодействий.

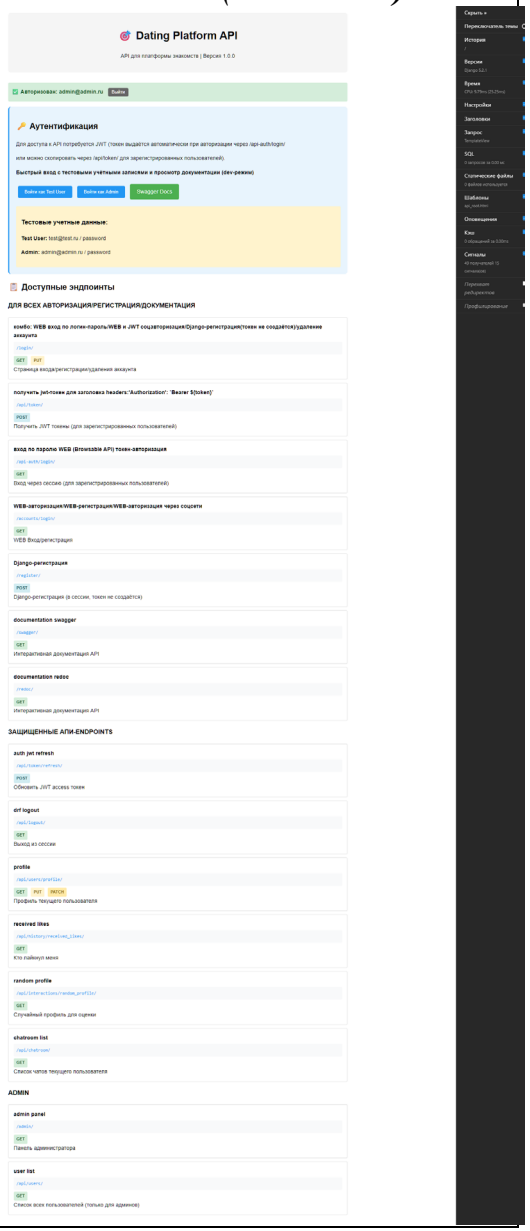
https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

I. Демонстрация работы API.

1. Главная страница (/)
Общедоступные эндпойнты (пользователь не зарегистрирован



2. Главная страница (/)
все эндпойнты (для админа)

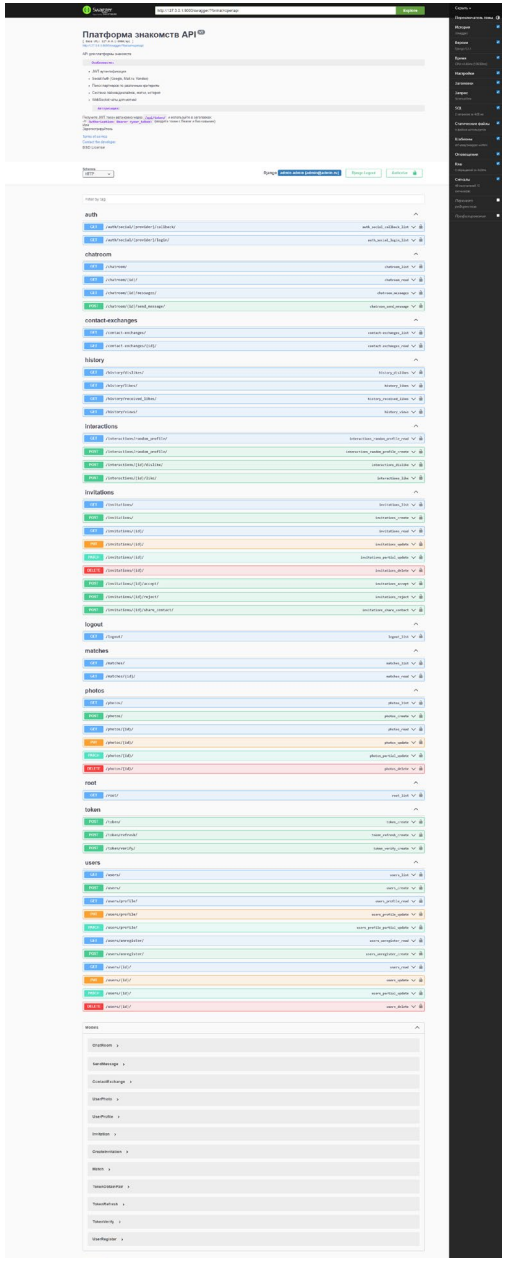


https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

3. Вход/регистрация (/login/)

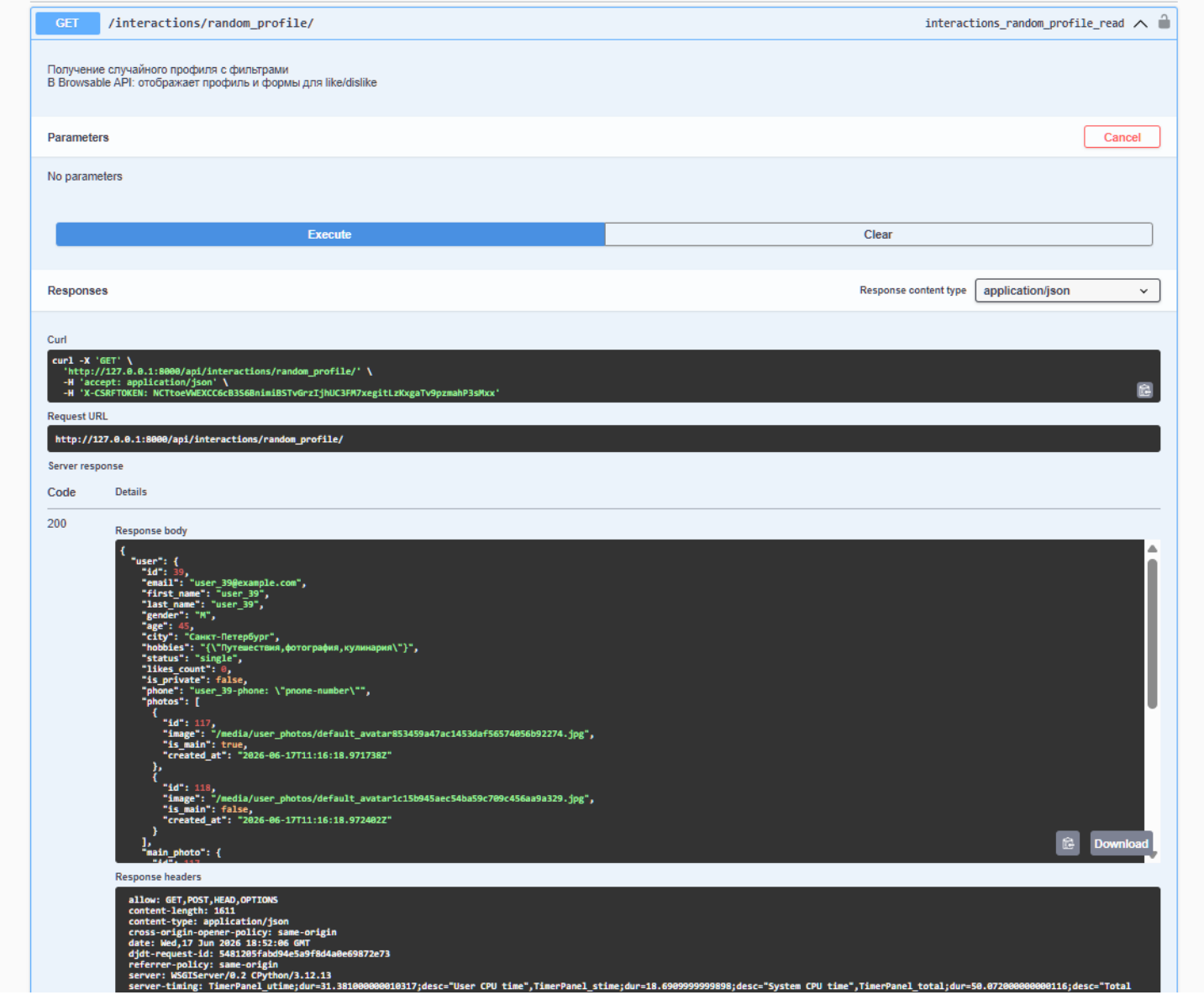


4. Документация Swagger UI (/swagger/



https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

5. Документация эндпоинта получения случайного профиля (/swagger).



GET /interactions/random_profile/ interactions_random_profile_read

Получение случайного профиля с фильтрами
В Browsable API: отображает профиль и формы для like/dislike

Parameters Cancel

No parameters

Execute Clear

Responses Response content type application/json

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/interactions/random_profile/' \
  -H 'accept: application/json' \
  -H 'X-CSRFTOKEN: NCTtoeVWEXCC6cB3S6BnimiBSTvGrzIjhUC3FM7xegitLzKxgaTv9pzmahP3sMxx'
```

Request URL

```
http://127.0.0.1:8000/api/interactions/random_profile/
```

Server response

Code Details

200

Response body


```
{
  "user": {
    "id": 39,
    "email": "user_39@example.com",
    "first_name": "user_39",
    "last_name": "user_39",
    "gender": "M",
    "age": 45,
    "city": "Санкт-Петербург",
    "hobbies": "{\"Путешествия,фотография,кулинария\"}",
    "status": "single",
    "likes_count": 0,
    "is_private": false,
    "phone": "user_39-phone: \"pnone-number\"",
    "photos": [
      {
        "id": 117,
        "image": "/media/user_photos/default_avatar853459a47ac1453daf56574056b92274.jpg",
        "is_main": true,
        "created_at": "2026-06-17T11:16:18.971738Z"
      },
      {
        "id": 118,
        "image": "/media/user_photos/default_avatar1c15b945aec54ba59c709c456aa9a329.jpg",
        "is_main": false,
        "created_at": "2026-06-17T11:16:18.972402Z"
      }
    ],
    "main_photo": {
```


Download

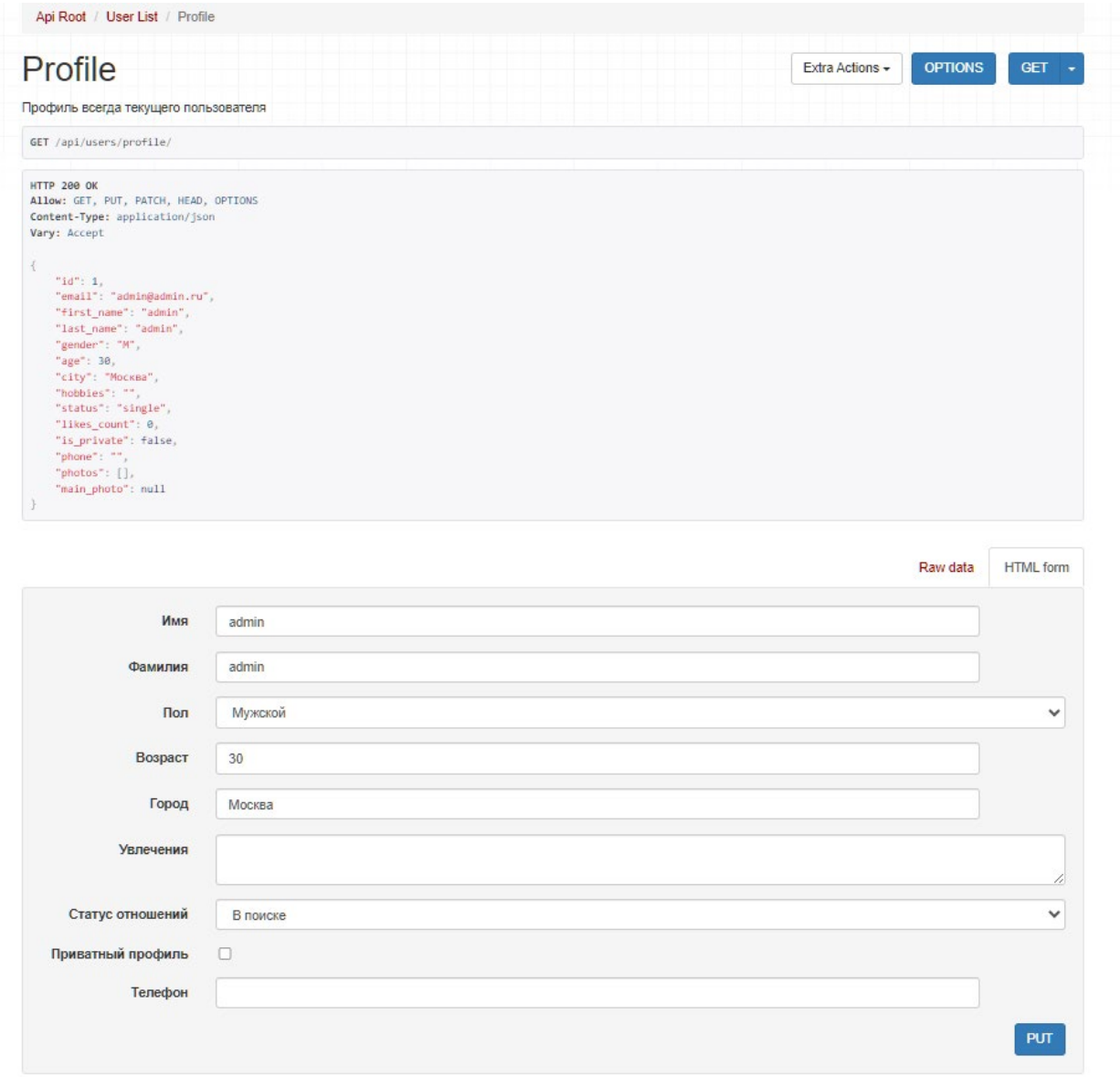
Response headers

```
allow: GET,POST,HEAD,OPTIONS
content-length: 1611
content-type: application/json
cross-origin-opener-policy: same-origin
date: Wed,17 Jun 2026 18:52:06 GMT
djdt-request-id: 5481205fabd94e5a9f8d4a0e69872e73
referrer-policy: same-origin
server: WSGIServer/0.2 CPython/3.12.13
server-timing: TimerPanel_utime;dur=31.381000000010317;desc="User CPU time",TimerPanel_stime;dur=18.6909999999898;desc="System CPU time",TimerPanel_total;dur=50.072000000000116;desc="Total
```

https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

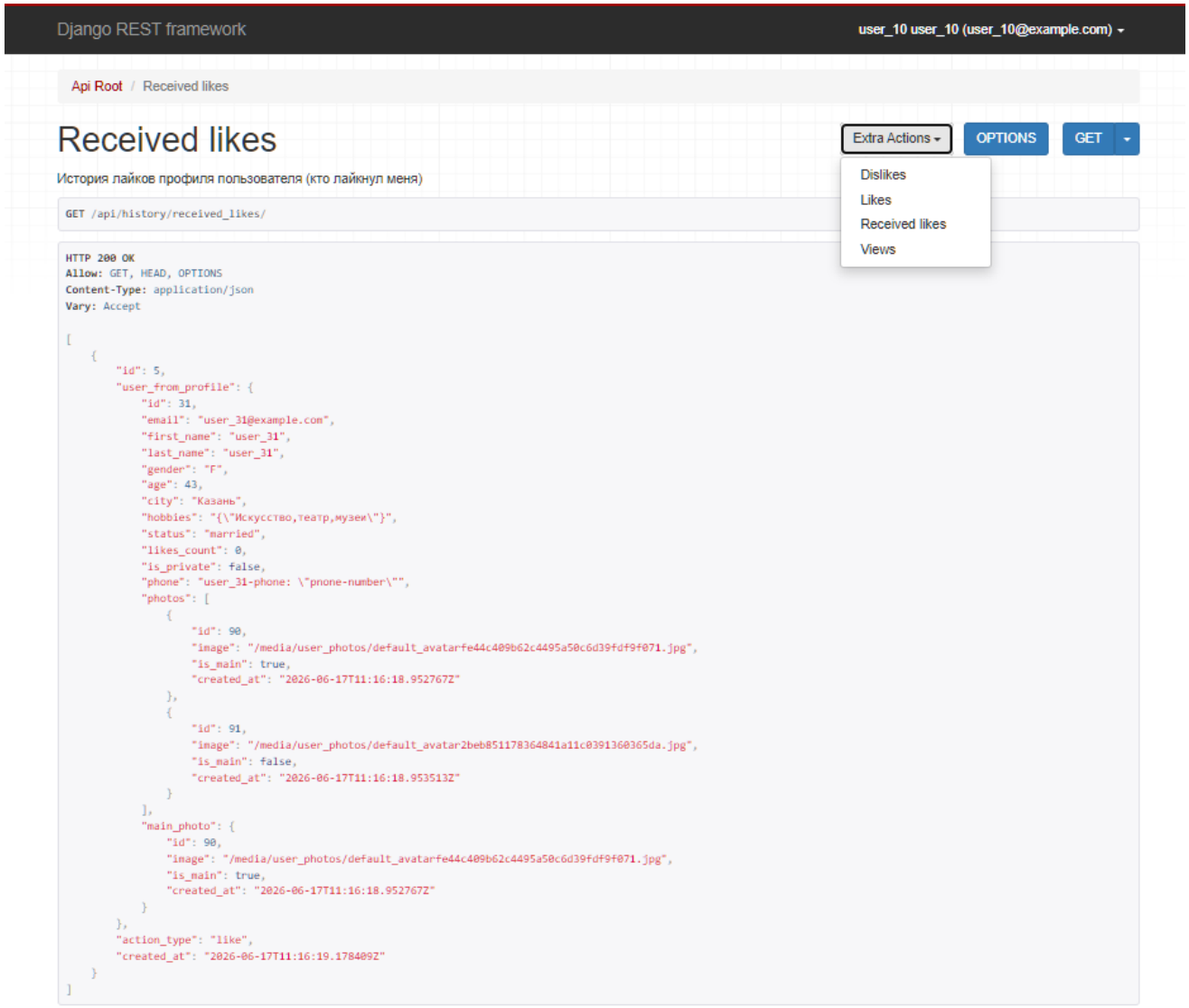
6. Примеры ответа API.

Эндпоинт получения профиля пользователя (всегда текущего). Browsable API(/api/users/profile/).



Эндпоинт полученных лайков. Browsable API(/history/received_likes/).

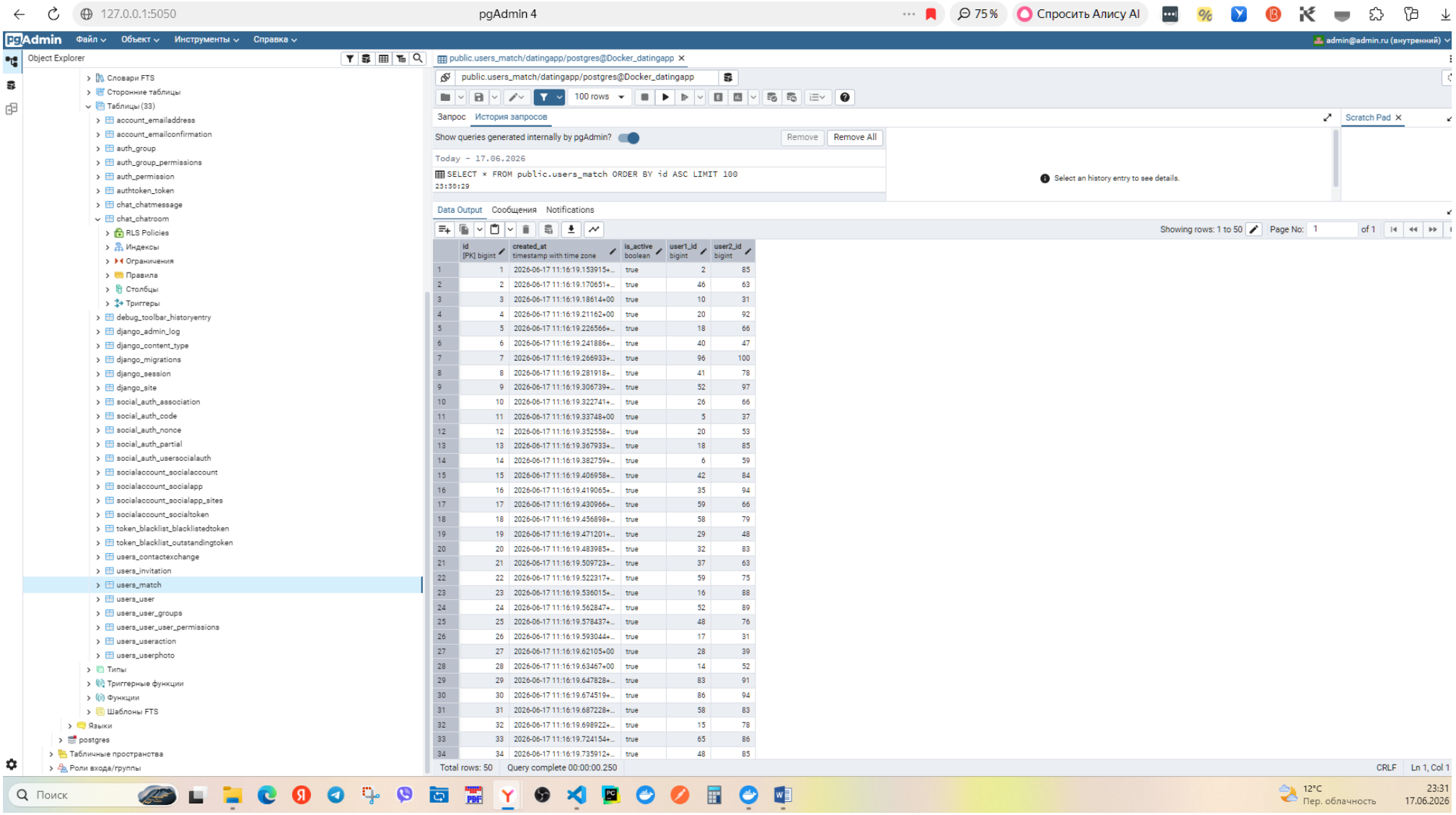
https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf



https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

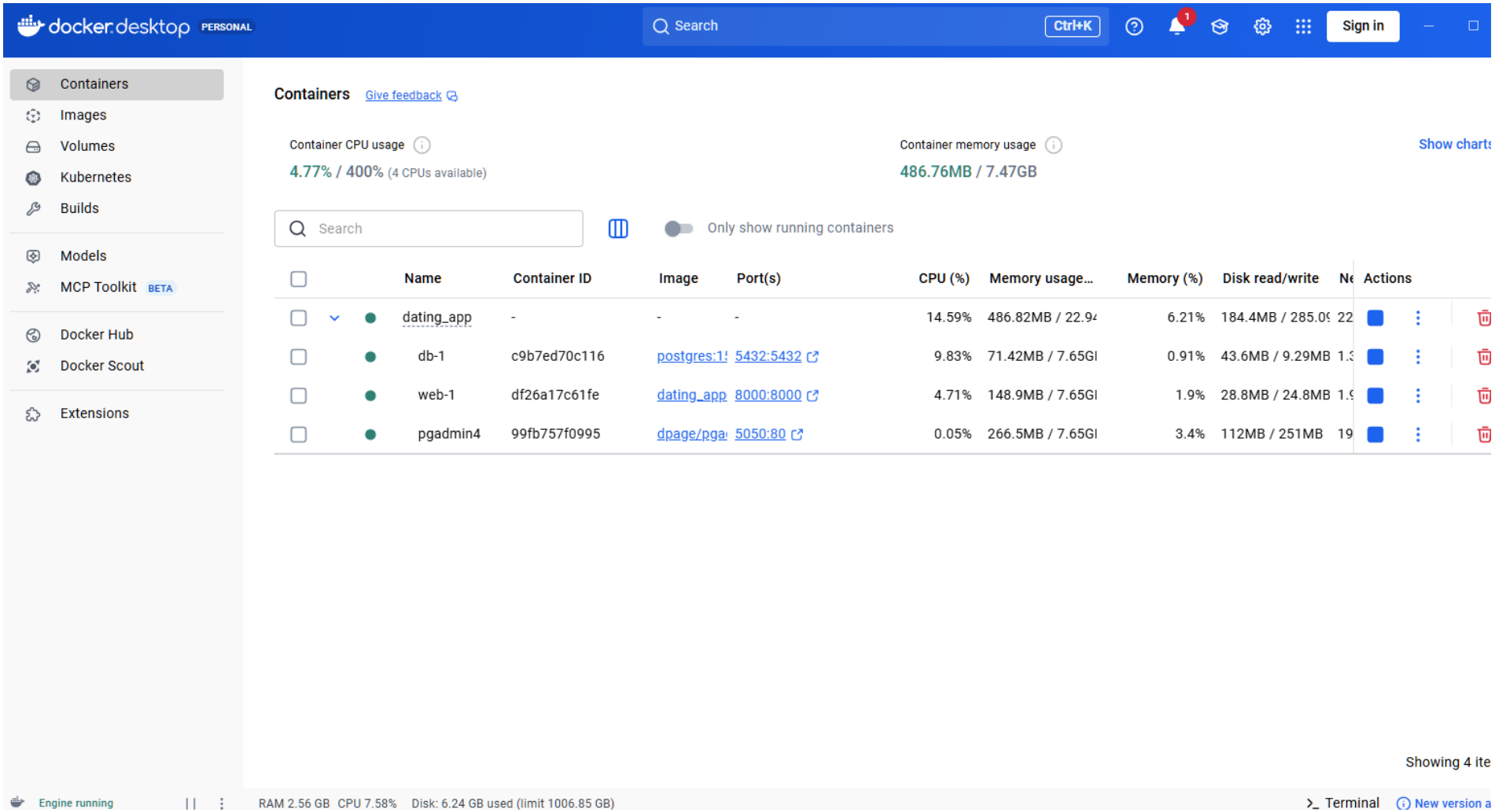
II. Интерфейс pgAdmin 4, развернут при запуске docker-контейнеров проекта.

Администрирование базы данных PostgreSQL(http://127.0.0.1:5050).



https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

III. Запуск проекта в изолированной среде с помощью Docker Compose.



https://github.com/EvgenyXIV/portfolio/blob/main/Кручинин_Евгений_hh_портфолио.pdf

